



Senpai - Weekly Progress Report

Phase 2 | Week 3

Ritankar Mondal | Abhay Chandra

貫いた設計原則

Held unchanged from Weeks 1–2

決定論ファースト

Deterministic first

全ての数値はPythonで算出。統計・ID解決・引用ファイアウォールは決定論的。

Every number is computed in pure Python — stats, ID resolution, citation firewall.

モデル無しでも正しく、有りでより良く

Correct without the model, better with it

プランナー・セグメント・整理機能はGPU無し/モデル停止時も動作。

Deterministic fallbacks; runs GPU-free when the served model is down.

回帰ゼロの移行

Zero-regression migration

各移行はパリティ試験で同一出力を証明してから旧経路を撤去。

Each migration was gated by a parity suite before the old path was retired.

モデルはIDや数値を創作しない

The model never invents an ID or a number

プランナーは「能力」を選ぶだけ。統計に無い数値を含む文章は棄却。

The planner picks capabilities, not IDs; ungrounded numbers are rejected.

根拠がある、さもなければ沈黙

Grounded or silent

記憶は実在エンティティに紐づく「引用付き所見」のみを保存。

Memory persists only cited observations anchored to a real entity id.

オーケストレーション・エンジン

The reusable spine — write one class, register it



1つの共通スパインが Research ・ Crew ・ Account ・ Chat の全てを駆動する。

One spine drives Research, Crew, Account and the Chat tool-loop — new capabilities become additive.

- **実行時に拡張されるDAG (ctx.expand)**

The DAG expands at runtime — N found files append N extract tasks

- **能力は単一領域を担当し、推論も他能力の呼び出しもしない**

Capabilities own one domain — they never reason or call each other

- **エビデンスは不変・追記のみ・出所を常に保持**

Evidence bundle is immutable, append-only, provenance always preserved

- **部分的失敗は劣化するがクラッシュしない**

Partial failure degrades gracefully — the run always completes

移行マイルストーン M0 – M6

Parity-proven, zero-regression migration

M0	エンジン本体を単体構築・テスト The engine itself — GPU-free, unit-tested in isolation
M1	Research を移行 (84ケースのパリティ) Research migrated — 84 parity cases, identical evidence bundles
M2	Crew + チーム展開を移行、並列化で高速化 Crew + team fan-out — tools now run in parallel (a latency win)
M3	Account 収集を移行 (123ケース: 60社×日英) Account gather — 123 cases (60 accounts × ja/en)
M4	Chat ループをエンジン統合 (AdaptiveScheduler) Chat tool-loop on the engine — READ ops auto-batched in parallel
M5	ワークスペース能力 = 初のシードDB外 + 実行時拡張 Workspace capability — first outside the seed DB, runtime expansion
M6	LLMプランナー: 目標 → 能力グラフ → 成果物 LLMPlanner — goal → capability graph → artifact

LLMプランナー

Document generation as a capability graph

「村田印刷の提案書を作って」 — スラッシュコマンド不要。目標が能力グラフになる。

Plain “make a proposal for Murata Printing” just works — the goal becomes an explicit capability graph.

- モデルは「能力」を選ぶ、IDは決定論的コードが解決
The model picks capabilities; deterministic code resolves identity (never the wrong deal)
- モデル停止時も正しい (heuristic_selection にフォールバック)
Correct without the model — heuristic fallback, proposal path fully GPU-free
- 通常チャットに統合、既存のUIイベントを再利用
Wired into normal chat — reuses the same plan/context/tool/document events
- 設計上ミニマル: 自律でも再帰でもない (1目標 → 1計画 → 1成果物)
Minimal by design — not autonomous, not recursive: one goal → one plan → one artifact

ローカル・ワークスペース・エージェント

Senpai steps outside the seed database

初めてユーザーの実ファイル进行操作 — 検索・読取・ノート生成・整理 (全てサンドボックス内)。

The first capability to act on the user's real local files — all sandboxed to WORKSPACE_ROOT.

- **検索・読取: PDF / DOCX / PPTX / XLSX を関連度順に抽出**

Search & read — relevance-ranked extraction across PDF/DOCX/PPTX/XLSX, cited as file://

- **ノート生成: 収集した根拠から Markdown ノートを作成**

Synthesize — authors a markdown note from the gathered grounding

- **整理: 散らばったファイルをトピック別フォルダへ**

Organize — tidies loose files into topic folders (quotes/proposals/...)

- **安全第一: サンドボックス脱出は禁止、削除操作は存在しない**

Safety first — sandbox escapes rejected, no delete op by design (unit-tested)

- **破壊的操作は2ターンのレビュー確認**

Two-turn preview confirm — lists moves first, applies only on “go ahead / はい”

賢いツール呼び出しとループ防止

A ReAct loop that provably can't spiral

- **finish センチネル: 収集後に即座にループを抜け、無駄なターンを節約**

"finish" sentinel — breaks the loop instantly once evidence exists

- **反復防止キャップ: 非生産的なラウンドのみを数える**

Anti-spiral cap — counts unproductive rounds only, never distinct-entity lookups

- **終端アクションの即時停止で成果物の重複を防止**

Terminal-action hard stop — prevents duplicate documents

- **完全重複排除 + 境界を意識した切り詰め (数値や社名を割らない)**

Exact dedup + boundary-aware truncation — never cuts a ¥ figure or company name in half

複数エンティティ展開: 「D133, D012, D168 を比較」

Multi-entity expander — 3 sequential rounds collapsed into 1 parallel round

75秒 → 28秒

実測レイテンシ、3案件を完全収集
measured latency, 3 deals fully gathered

文脈と記憶の管理

Grounding, focus, and cross-chat memory

「さっき話した会社の資料を作って」 — 背景スレッドが会話を正しく共有する。

Fixes the failure where a background thread can't see the chat and hallucinates a generic deck.

- スレッド安全な会話グラウンディング (ContextVar で並列共有)

Thread-safe conversation grounding — ContextVar so concurrent users never cross-contaminate

- 再現性ではなく関連性で選択 (CJK bigram でスクリプト非依存)

Relevance not just recency — script-agnostic CJK bigrams (村田印刷 → 村田/田印/印刷)

- SessionFocus: 実ツール結果の明確なIDに紐づく (曖昧名を排除)

SessionFocus — keyed off unambiguous IDs from tool results, never fuzzy names

- チャット横断記憶: 会話履歴ではなく「引用付き所見」を保存

Cross-chat memory — persists cited Observations (not transcripts), anchored by entity + timestamp

二段階リーズナー & 二重モデル基盤

Citation firewall · failover · token accounting

二段階リーズナー

Two-pass reasoner — citation firewall

- **解釈: エビデンスを「引用付き所見」に変換**
Interpret — evidence becomes typed, cited observations (temp 0, JSON)
- **引用が根拠に遡れない所見は破棄**
Any observation whose citations don't trace to evidence is dropped
- **構成: 重要度ランク順に成果物を執筆**
Compose — authors from ranked observations (materiality high/med/low)
- **失敗時は単発合成にフォールバック (回帰なし)**
Falls back to single-shot synthesis — no regression

ハイブリッド二重モデル

Dual-model client — failover & tokens

- **プライマリ障害時に自動フェイルオーバー**
Automatic failover — retries on the fallback server, no crashed turn
- **トークン計上を1行JSONLで記録**
Token accounting — one JSONL row per completion, shown on the admin Usage page
- **推定値は明示ラベル、推測を計測値として提示しない**
Estimates are clearly labelled — never presented as measured

セグメント・インテリジェンス

GraphRAG community summarization for managers

「製造業のサーバー案件、なぜ負ける？」 — マネージャーの全体・横断的な問いに答える。

Answers the global, aggregate question a manager asks — which local retrieval could never do.

- **GraphRAGの高コストな抽出工程(step1)を省略 — 既に構造化グラフを保有**

Skips GraphRAG's expensive LLM extraction — we already have a clean typed graph

- **コミュニティ = 決定論的ファセット (製品分類 × 業種)**

Communities are deterministic facets (category × industry) — no Leiden needed

- **決定論的な数値 + LLMの文章 + 検証された文章**

Deterministic numbers, LLM prose, verified prose — a grounding gate rejects any invented figure

- **レポートは事前計算・コミット、実行時は「縮約」を無料で**

Reports precomputed & committed; the chat loop's synthesis round does the "reduce" for free

- **構造上、存在しない数値を提示できない**

Structurally cannot surface a hallucinated number (machine-checked grounding invariant)

製品基盤の強化

The plumbing a real deployment needs

- **認証 (サインアップ / ログイン) — アカウントをシード担当者に紐付け**

Auth — signup/login maps accounts onto seed reps (junior→rep, manager→coach)

- **永続チャット履歴 (SQLite) + 履歴ドロワー + ライブ検索**

Persistent chat history — SQLite store + History Drawer with live search

- **ジュニア・コマンドセンター: 6ページを1画面に統合**

Junior Command Center — six pages collapsed into one split-pane home

- **マネージャー・コマンドセンター: チーム・トリアージのペイン**

Manager Command Center — team-triage pane grounds the Copilot on click

- **リアルタイムTTS + 一元化された設定モジュール**

Real-time TTS + central config module (env-based tunables, nothing hardcoded)

- **UI連携: ClientBadge/ContextPane 統合、型付きAPIブリッジ、旧認証の整理**

UI polish — ClientBadge/ContextPane integration, typed API bridge, legacy auth cleanup

次のステップ

The next milestone is expansion, not rewrite

同じスパインの上に、リーズナーと縮約を加えて長時間タスクへ拡張する。

The same spine plus a real Reasoner pass and a Reducer — no rewrite.

- **会議準備 / アカウント・インテリジェンスヘプランナーを拡張**

LLMPlanner for meeting-prep & account-intelligence — “prepare tomorrow’s Endo Kogyo meeting”

- **チャット横断記憶の読み取り側を接続**

Wire cross-chat memory read-side — inject observations into grounding (token-capped)

- **（後回しにした簡素化）4つのリーズナーを reason.py に集約、SSE方言を統一**

Deferred simplification — converge reasoners onto reason.py, unify SSE dialects

- **承認ゲート + 外部連携 (メール / カレンダー)**

Approval Gate + ConnectionProvider — per-user auth for external Email / Calendar